ESP32 Emulator

Universal ESP32 sketch emulator with virtual oscilloscope

Run Arduino C++ sketches (.ino) or Python scripts, inspect the voltage of every GPIO pin, zoom into waveforms and test peripherals without hardware.

Files: ESP32_Simulator.py (GUI + chip model), esp32_cpp.py (C++ to Python translator)

1. Overview

The program emulates an ESP32 at the **behavioural level**: you enter source code, it is executed against a model of the chip (GPIO, ADC, DAC, PWM, timers, interrupts, UART, I2C, FreeRTOS, ...), and the resulting pin voltages are shown on an oscilloscope. It is not an instruction-level (CPU) emulator: the sketch itself runs as Python, while all peripherals react like the real hardware and carry real timing (an emulated clock).

1.1 What you can do

- Load any Arduino-style **C++ sketch** (`setup()` / `loop()` or `app_main()`) - it is translated automatically.
- Or write the same API directly in **Python**.
- Select **any GPIO** and watch its voltage over time (digital levels, PWM, DAC/analog, floating/pulled, externally driven).
- **Zoom** the time axis with the mouse wheel (down to 100 ns for the whole window), pan by dragging.
- **Drive** an input pin with a slider (0-3.3 V) to feed ADC, buttons or interrupts.
- Talk to the sketch over **Serial** (console + input line), see the UART waveform on the TX pin.
- Run heavy sketches (e.g. 100 kHz PWM interrupts) in **slow motion** using the speed control.

1.2 Requirements

- Python 3.9 or newer with tkinter (included with the standard Windows installer).
- **numpy** (optional but recommended): used for exact, alias-free rendering of dense waveforms. Install with `pip install numpy`. Without it a coarser fallback is used.
- Both files must be in the same folder: `ESP32_Simulator.py` and `esp32_cpp.py`.

1.3 Starting

```
python ESP32_Simulator.py
```

The program opens with a built-in example sketch. Press **Run** to execute it.

2. User interface

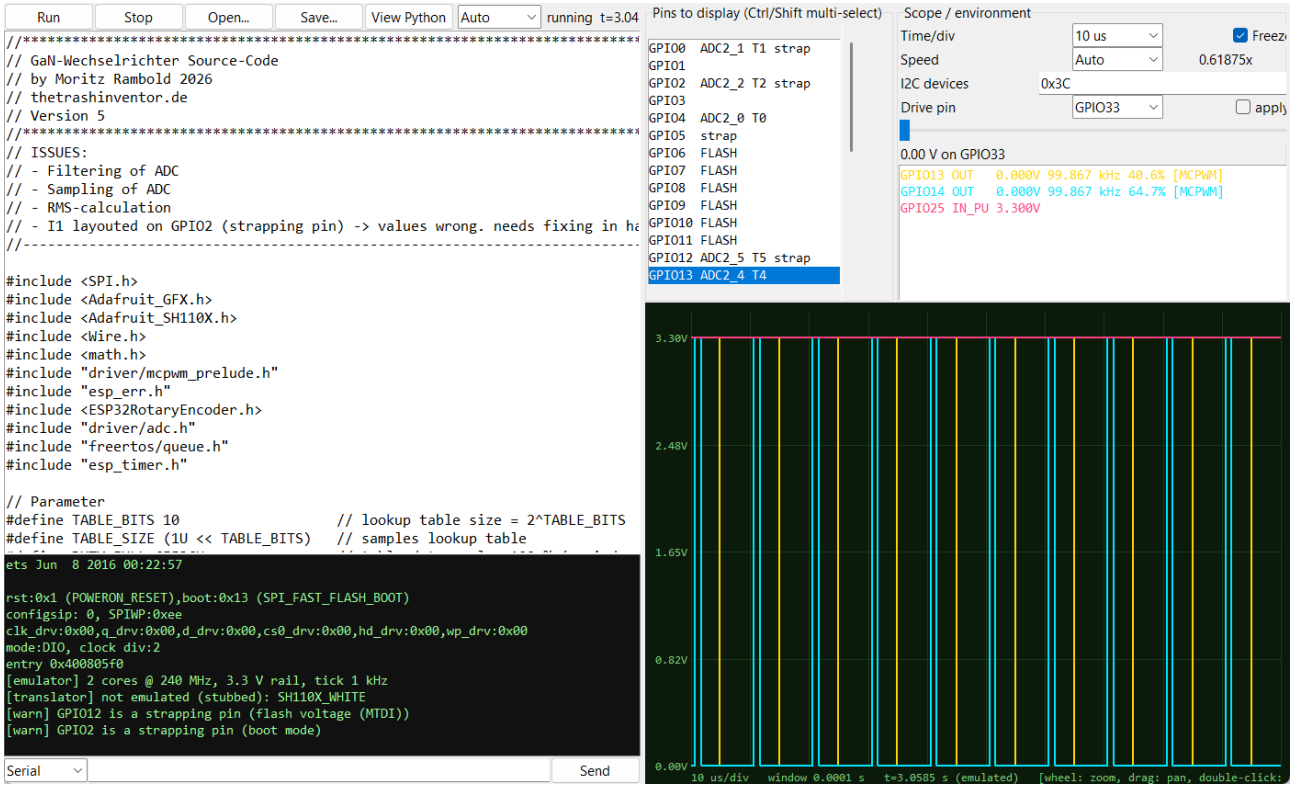


Figure 1: Emulator running the GaN inverter sketch (MCPWM at 99.87 kHz) with the scope at 10 us/div.

Area	Function
Toolbar	**Run** compiles/starts the code, **Stop** halts it, **Open / Save** load or save a sketch file, **View Python** shows the generated Python for a C++ sketch. The language box selects *Auto* (detected), *C++* or *Python*. The status text shows emulated time, speed and running tasks.
Editor (left top)	Source code. C++ is detected by `void setup()/loop()/app_main()`.
Console (left bottom)	Serial output (Serial, plus Serial1/Serial2 prefixed with [UARTn]), boot banner, [warn] messages, compile errors and 'Guru Meditation' panics with the failing line.
Serial input	Choose Serial/Serial1/Serial2 and press **Send** (or Enter): the text plus newline is fed into `Serial.available()/read()`.
Pins to display	Multi-select list (Ctrl/Shift) of all usable GPIOs with function hints (ADC channel, touch, strapping, FLASH). Each selected pin is drawn in its own colour.
Scope / environment	**Time/div**, **Freeze**, **Speed**, **I2C devices**, **Drive pin** with slider and **apply** checkbox. Below it a legend lists every shown pin.
Scope (right bottom)	The oscilloscope: 10 divisions horizontally, 0-3.3 V vertically. See chapter 3.

2.1 Legend line

Example: `GPIO13 OUT 0.000V 99.867 kHz 40.6% [MCPWM]` = pin, mode (IN, OUT, IN_PU, IN_PD, OD), voltage at the right scope edge, measured frequency and duty cycle, and the peripheral that owns the pin (MCPWM, UART, I2C, ...).

3. Oscilloscope

3.1 Time axis

Action	Effect
Mouse wheel on the scope	Zoom the time axis in/out. Live: the newest sample stays at the right edge. Paused: the time under the cursor stays fixed.
Drag with left button	Pan back through history. The view pauses automatically (Freeze ticks).
Double-click	Return to the live view.
Time/div box	Set the window directly: 1 us ... 1 s per division (10 divisions).
Freeze	Pause the display (the sketch keeps running).

History is limited to 300,000 edges per pin; for a 100 kHz signal this is about 1.5 s of data.

3.2 Rendering

- **Sparse signals** are drawn exactly, edge by edge (polyline). PWM from LEDC is expanded per period when at least 3 pixels per period are available.
- **Dense signals** (more than about 3000 edges in the window, e.g. SPWM) are drawn without aliasing: for every pixel column the exact time-weighted **average** is calculated (this shows the sine envelope) and a translucent **min/max band** shows that edges exist (peak detect, like a real digital scope). Even a 20 ns glitch is visible.
- Vertical scale: 0 ... 3.3 V with grid at 0.825 V steps.

3.3 Driving inputs

Select a pin in **Drive pin**, move the slider and tick **apply**. The pin is then driven externally with that voltage: `analogRead()` returns the matching ADC value, `digitalRead()` uses the VIL/VIH thresholds (a pin must exceed about 0.75 x 3.3 V to read HIGH) and interrupts fire on the edges. Un-tick **apply** to release the pin.

4. Speed and timing model

The emulator has its own **emulated clock**. `millis()`, `micros()`, `delay()`, timers and PWM all use it. The **Speed** setting maps emulated time to real time:

Setting	Meaning
1x	Real time. Fine for ordinary sketches.
0.1x / 0.01x / 0.001x / 0.0001x	Slow motion: 1 s emulated takes 10 / 100 / 1000 / 10000 s. Use for very fast events (e.g. 100 kHz MCPWM interrupts) which Python cannot keep up with in real time.
Auto (default)	Starts at 1x and lowers the speed whenever the event engine falls behind (real-time lag above 3 ms); after a calm period it raises the speed again in small steps. After every change it waits 1 s before judging again.

Because waveforms are stored with emulated timestamps, the scope shows exactly the same signal at any speed - only the wall-clock duration changes. The speed achieved depends on your PC and on how many events the sketch generates (a 100 kHz, three-channel inverter sketch runs at roughly 0.5-1x on a typical PC).

4.1 Execution model

- **loopTask** (core 1) runs `setup()` once and then `loop()` repeatedly, like the Arduino core.
- **Timer/ISR engine** (core 0 context) executes interrupts, hardware timers, esp_timer callbacks and MCPWM cycles at their exact emulated times from a time-ordered event queue.
- **FreeRTOS tasks** created with xTaskCreate / xTaskCreatePinnedToCore run as separate threads; the tick rate is 1 kHz (pdMS_TO_TICKS(x) = x).
- A crash in a task prints *Guru Meditation Error* with the exception and the line in the generated code, like a real panic; other tasks keep running.

5. Emulated hardware

Peripheral	What is emulated
GPIO	Pins 0-5, 12-19, 21-23, 25-27, 32-39. 34-39 are input-only; 6-11 are flash pins (use is reported as an error); strapping pins (0, 2, 5, 12, 15) give warnings. Pull-up/down, open drain, VIL/VIH thresholds, `attachInterrupt` RISING / FALLING / CHANGE.
ADC	12-bit, attenuation ranges, noise, clipping; `analogRead`, `analogReadMilliVolts`, ADC1/ADC2 channel mapping.
DAC	8-bit on GPIO25/26 (`dacWrite`); shown as an analog level.
Touch	`touchRead(T0..T9)`.
LEDC PWM	16 channels, frequency/resolution limits (80 MHz timer: invalid combinations produce an error as on the chip), Arduino core 2.x (`ledcSetup/ledcAttachPin/ledcWrite(channel)`) and 3.x (`ledcAttach/ledcWrite(pin)`), `analogWrite`, `tone`.
MCPWM	ESP-IDF 5 API: timer, operator, comparator, generator, up and up-down counting, update on TEZ, on-empty / on-full callbacks, generator actions at exact tick times (e.g. complementary GaN half-bridge).
Timers	Hardware timers (Arduino 2.x and 3.x API) and `esp_timer` one-shot / periodic.
FreeRTOS	Tasks on 2 cores, delays, queues, semaphores/mutexes (out-parameters via &handle supported).
UART	Serial (TX GPIO1), Serial1 (GPIO10), Serial2 (GPIO17): bit-exact 8N1 waveform on the TX pin with FIFO back-pressure.
I2C	`Wire` master with virtual register-file devices. Addresses are set in the **I2C devices** field (e.g. `0x3C, 0x68`). An absent address makes `endTransmission()` return 2 (NACK), so I2C scanners work.
SPI	Basic `SPIClass` transfers (no external SPI device models).
Misc	`millis, micros, delay, delayMicroseconds, random, map, constrain, bit*, ESP.getFreeHeap`-style helpers, ESP-IDF error handling (ESP_ERROR_CHECK, esp_err_t).

5.1 Not emulated

- Wi-Fi, Bluetooth, USB, Ethernet, SD card, I2S, RMT, CAN, camera.
- Instruction-level timing: code between hardware events takes (almost) no emulated time; use delay/timers for timing.
- Analog circuit behaviour beyond the stated pin models (no external components, no loads).

6. C++ sketches

C++ is translated to Python by `esp32_cpp.py` (preprocessor, tokenizer, type-aware translator) and executed. Use **View Python** to inspect the result. C semantics are preserved where it matters: truncating integer division and modulo, unsigned and fixed-width wrap-around (uint8_t, uint32_t, ...), integer vs. float arithmetic, casts, `switch` with fall-through, `do/while`, `static` locals, structs, enums, arrays, `String` and `char[]` (as text), `printf`-style formatting, `#define` macros (object-like and function-like) and `#if/#ifdef`.

6.1 Supported

- Functions, recursion, global/local/static variables, pointers used as out-parameters (`&x`), arrays and pointers to arrays.
- Library qualifiers such as IRAM_ATTR, ARDUINO_ISR_ATTR, volatile, const, PROGMEM.
- Serial.print / println (with base or decimals) / printf / write / available / read.
- Unknown library types become dummy objects that accept any call; the console lists them as *[translator] not emulated*.

6.2 Not supported (compile error with line number)

- Classes with methods, templates, lambdas, new/delete, goto, namespaces.
- References/pointers to plain scalars are passed by value (except address-taken variables, which work).
- Third-party library objects (e.g. OLED displays, rotary-encoder classes) are dummies: calls succeed but do nothing.

6.3 Example: MCPWM complementary output

```
#include "driver/mcpwm_prelude.h"
mcpwm_timer_handle_t t; mcpwm_oper_handle_t o; mcpwm_cmpr_handle_t c; mcpwm_gen_handle_t ga;
void setup(){
  mcpwm_timer_config_t tc = {}; tc.group_id=0; tc.clk_src=MCPWM_TIMER_CLK_SRC_DEFAULT;
  tc.resolution_hz=10000000; tc.count_mode=MCPWM_TIMER_COUNT_MODE_UP; tc.period_ticks=500;
  mcpwm_new_timer(&tc,&t);
  mcpwm_operator_config_t oc = {}; oc.group_id=0;
  mcpwm_new_operator(&oc,&o); mcpwm_operator_connect_timer(o,t);
  mcpwm_comparator_config_t cc = {}; cc.flags.update_cmp_on_tez=true;
  mcpwm_new_comparator(o,&cc,&c); mcpwm_comparator_set_compare_value(c,150);
  mcpwm_generator_config_t gc = {}; gc.gen_gpio_num=13; mcpwm_new_generator(o,&gc,&ga);
  mcpwm_generator_set_action_on_timer_event(ga,
     MCPWM_GEN_TIMER_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP,
                                  MCPWM_TIMER_EVENT_EMPTY, MCPWM_GEN_ACTION_HIGH));
  mcpwm_generator_set_action_on_compare_event(ga,
     MCPWM_GEN_COMPARE_EVENT_ACTION(MCPWM_TIMER_DIRECTION_UP, c,
                                    MCPWM_GEN_ACTION_LOW));
  mcpwm_timer_enable(t); mcpwm_timer_start_stop(t, MCPWM_TIMER_START_NO_STOP);
}
void loop(){ delay(1000); }
```

Select GPIO13 and set Time/div to 10 us: the legend shows 20 kHz and 30 % duty.

7. Python sketches

The same functions and constants are available directly. Define `setup()` and `loop()` (or `app_main()`); no imports are needed.

```
def setup():
    pinMode(2, OUTPUT)
```

```
    Serial.begin(115200)

def loop():
    digitalWrite(2, HIGH); delay(250)
    digitalWrite(2, LOW);  delay(250)
    Serial.println(analogRead(34))
```

8. Typical workflows

Goal	Steps
Check a PWM output	Load sketch, Run, select the pin, adjust Time/div (or wheel-zoom). Read frequency and duty in the legend.
Test a button / interrupt	Select the button pin in Drive pin, tick apply and move the slider below ~0.8 V (pressed, for INPUT_PULLUP) and back above 2.5 V; watch the Serial counter.
Read a sensor value	Drive an ADC pin (e.g. GPIO34/35/36/39) with the slider; the sketch's analogRead follows. 1.95 V reads about 2040 at 11 dB attenuation.
Inspect fast waveforms	Let Auto speed settle, press Freeze, then zoom with the wheel and pan by dragging.
Debug a crash	Read the Guru Meditation line in the console, then **View Python** and look at the given line of generated code.
Simulate an I2C device	Enter its address in the I2C devices field before Run, e.g. `0x3C, 0x68`.

9. Architecture

Component	Description
esp32_cpp.py	preprocess (#define/#if) -> tokenize -> macro expand -> two-pass Translator (signatures, structs, enums, address-taken variables, then code emission). Entry points: `transpile(src, known)` and `looks_like_cpp(src)`.
Clock	Emulated time with an adjustable scale (speed).
Engine	Thread with a time-ordered event heap: interrupts, timers, esp_timer, MCPWM cycles. Periodic events are re-queued; events due within a few ms of real time are processed in one batch.
Pin	Timeline of (time, state) with kinds L (level), P (LEDC PWM), A (analog), E (external), Z (floating).
Peripherals	McTimer/McOper/McCmp/McGen, HwTimer, EspTimer, HardwareSerial, TwoWire (+I2CDevice), SPIClass, FreeRTOS objects.
ESP32 class	Builds the namespace (functions, constants) the sketch sees, starts tasks, handles errors and the boot banner.
App (tkinter)	Editor, console, pin list, controls, oscilloscope renderer (numpy-accelerated), zoom/pan handlers.

10. Troubleshooting

Symptom	Cause / remedy
[compile error] line N: ...	Unsupported or malformed C++ at that line (see 6.2). Rewrite that construct or use Python.
Many *[translator] not emulated* names	Library calls not modelled (Wi-Fi, display, etc.). They are stubbed and ignored.
Display is laggy / speed drops	Too many events. Select fewer pins, use a larger Time/div, or set a fixed slow speed.
[warn] GPIOn is a strapping pin	Real-chip warning: the pin influences boot mode; avoid it for loads that pull it at reset.
Error on flash pins 6-11	These pins are connected to the SPI flash on real modules and must not be used.
No output on Serial	Call `Serial.begin()`; for Serial1/2 select the matching port in the input box.
Zoom shows no data on the left	History is limited (300,000 edges per pin); older data was discarded.

11. Validation

The emulator was verified with a set of classic sketches (all pass): Blink, millis-Blink, Fade (LEDC), AnalogReadSerial, button interrupt, hardware timer, FreeRTOS tasks on both cores, I2C scanner, DAC sine, analogWrite, touchRead, Serial echo, MCPWM 20 kHz generator, and a C-semantics test (recursion, sorting, structs, enums, switch fall-through, String, unsigned wrap-around, integer/float division, snprintf, map/constrain). In addition a 900-line GaN inverter sketch (3-channel 100 kHz MCPWM with lookup-table SPWM, esp_timer current sampling, FreeRTOS UI task) runs without errors and produces 99.87 kHz outputs with the expected duty cycles.